

Grado en Ingeniería Informática
2024-2025

Trabajo Fin de Grado

“Desarrollo de un dataset para el cálculo de distancias entre núcleos urbanos y servicios de ámbito comarcal con titularidad pública en la Comunidad de Madrid”

Raúl García Romero

Tutor/es

Javier García Guzmán

Leganés, 2025



Esta obra se encuentra sujeta a la licencia Creative Commons
Reconocimiento – No Comercial – Sin Obra Derivada

RESUMEN

El siguiente Trabajo de Fin de Grado está orientado a el cálculo de disponibilidad de distintos servicios públicos para los distintos municipios de la Comunidad de Madrid, a partir del cálculo de la distancia media entre cada núcleo urbano y un conjunto con cada uno de los siguientes servicios públicos: Hospitales, juzgados, centros de salud mental y parques de bomberos.

Inicialmente, los datos utilizados para llevar a cabo esta práctica son distintos datasets georreferenciados, es decir, un conjunto de instancias de datos cada una ligada a un punto geográfico por medio de coordenadas. Estos datos serán preprocesados, procesados y utilizados en programas de Python y QGIS para obtener distintos conjuntos de datos que den la información necesaria.

ÍNDICE DE CONTENIDOS

1.	INTRODUCCIÓN.....	1
1.1.	Área de Trabajo	1
1.2.	Problema por resolver.....	1
1.3.	Motivación del Trabajo.....	1
1.4.	Objetivos.....	1
2.	ESTADO DE LA CUESTIÓN	3
2.1.	Trabajos previos y documentación existente	3
2.2.	Noticias sobre disponibilidad de servicios públicos en Madrid	4
3.	MÉTODOS.....	5
3.1.	Fuente de los datos	5
3.2.	Procesamiento inicial de los datos mediante QGIS.....	6
3.3.	Procesamiento adicional mediante Python	7
3.4.	Cálculo de distancias utilizando la API de Google Maps.....	7
4.	METODOLOGÍA PARA LA CONSTRUCCIÓN DEL DATASET.....	8
4.1.	Obtención de los datos iniciales	8
4.2.	Procesado de datos en QGIS	9
	Filtrado de diseminados y núcleos con menos de 10 habitantes	9
	Obtención de los centroides.....	9
	Adición de atributos	13
4.3.	Selección de vecinos más cercanos a través de la librería de QGIS para Python	15
	Métodos <code>closest_destinations_cords()</code> y <code>closest_destinations_features()</code>	15
4.4.	Consulta a la Routes API de Google	17
	Métodos para la realización de consultas	18
4.5.	Depurado de los resultados.....	21
4.6.	Unión con el dataset en formato Geopackage	22
	Capa de centroides.....	23
	Capa de municipios	24
5.	DESCRIPCIÓN DEL DATASET OBTENIDO	25
6.	ESPECIFICACIÓN DE REQUISITOS DE LA APLICACIÓN GIS PARA VISUALIZAR LOS DATOS.....	26
7.	DISEÑO	27
8.	RESULTADOS	28
9.	CONCLUSIONES Y FUTUROS TRABAJOS.....	29

ÍNDICE DE FIGURAS

Ilustración 1 Captura de pantalla de la página web NOME CALLES	5
Ilustración 2 Vista previa de la herramienta QGIS	6
Ilustración 3 Vista previa del código usando la API de QGIS.....	7
Ilustración 4 Proceso de filtrado de núcleos poblacionales.....	9
Ilustración 5 Ajuste de centroide a carretera	10
Ilustración 6 Ajuste a parada de bus más cercana	11
Ilustración 7 Tabla de hospitales de grupo 2 y 3	12
Ilustración 8 Núcleos y hospitales clasificados por su DAS	13
Ilustración 9 División por partido judicial.....	14
Ilustración 11 Flowchart de funciones implementadas en Python .	¡Error! Marcador no definido.
Ilustración 12 Función get_cords_name()	¡Error! Marcador no definido.
Ilustración 13 Función get_nearest()	¡Error! Marcador no definido.
Ilustración 14 Función get_jsons()	¡Error! Marcador no definido.
Ilustración 15 Ejemplo de archivo de la carpeta jsons/ ..	¡Error! Marcador no definido.
Ilustración 16 Código de google_request.py	¡Error! Marcador no definido.
Ilustración 17 Ejemplo de output generado por una consulta a Distance Matrix API	¡Error! Marcador no definido.

ÍNDICE DE CÓDIGO

Código 1 Método <code>closest_destinations_features()</code>	16
Código 2 Método <code>closest_destinations_cords()</code>	16
Código 3 Método <code>request_routes_v2_async</code>	19

LISTA DE ABREVIATURAS

QGIS	Quantum Geographic Information System
DAS	Dirección Asistencial de Salud
INE	Instituto Nacional de Estadística

GLOSARIO

Término	Descripción
Núcleo de población	Conjunto de al menos diez edificaciones y/o cincuenta habitantes con calles, plazas u otras vías urbanas. [1]
Diseminado	Edificaciones o viviendas que no pueden ser incluidas en el concepto de núcleo de población. [2]

1. INTRODUCCIÓN

1.1. Área de Trabajo

El área de trabajo de este estudio son los 179 municipios que componen la Comunidad de Madrid, así como los núcleos poblacionales que lo forman y las distintas infraestructuras donde se brindan servicios públicos esenciales.

La información sobre este dominio de trabajo la tratamos en forma de datasets georreferenciados que nos permiten operar sobre ellos, obtener información adicional y permite la visualización de dichos datos en el mapa mediante aplicaciones como QGIS, ArcGIS

1.2. Problema por resolver

Un problema que genera bastante preocupación desde hace muchos años es la falta de servicios públicos en zonas rurales o alejadas de centros de provincia y ciudades importantes. A esto se le suma la falta de datos empíricos y demostrables que validen y cuantifiquen el grado de accesibilidad de los servicios públicos y cuáles son los servicios públicos más necesarios para reforzar en cada zona.

Por lo tanto, el problema que pretendemos abordar es el del desarrollo de una base de datos en la que se pueda ver la disponibilidad de servicios en todas las áreas de la Comunidad de Madrid

1.3. Motivación del Trabajo

El fácil acceso y cercanía a servicios públicos es una de las principales necesidades para el bienestar de los habitantes de una ciudad. En un mundo industrializado y con la infraestructura necesaria para conectar todos los núcleos urbanos de un país, es cada vez más fácil y necesario determinar qué servicios públicos brindar y dónde se establece la construcción de la infraestructura destinada a brindar dicho servicio.

Por lo tanto, se tiene como principal objetivo determinar el nivel de necesidad de nuevos centros públicos para cada núcleo urbano y, por lo tanto, de cada municipio.

1.4. Objetivos

Este trabajo tiene como meta la obtención de información y la subsecuente creación de una base de datos sobre la disponibilidad de distintos servicios públicos en los diferentes municipios de la Comunidad de Madrid

Los servicios públicos sobre los cuales se hace el trabajo serán:

- Hospitales
- Juzgados
- Centros de salud mental
- Parques de bomberos

Software utilizado: QGIS, software de gestión de bases de datos con representación geográfica de la información.

Fuente de los datos: nomecalles [3]. Base de datos con puntos de interés y distintas delimitaciones de la comunidad de Madrid

- Andrés Roca y Moreno-Jimenez Antonio en su trabajo “Accesibilidad de los jóvenes a la red de transporte público en Alcobendas (Madrid) por motivo de ocio nocturno” [6] hacen un estudio del grado de disponibilidad de transporte público en la zona de Alcobendas, teniendo en cuenta puntos clave como nivel de renta medio de la zona, tiempo de desplazamiento a bocas de Metro o paradas de bus y accesibilidad a otras zonas de ocio. En general, muestra la gran cantidad de factores que forman parte de lo que entendemos como “disponibilidad” de un servicio público.

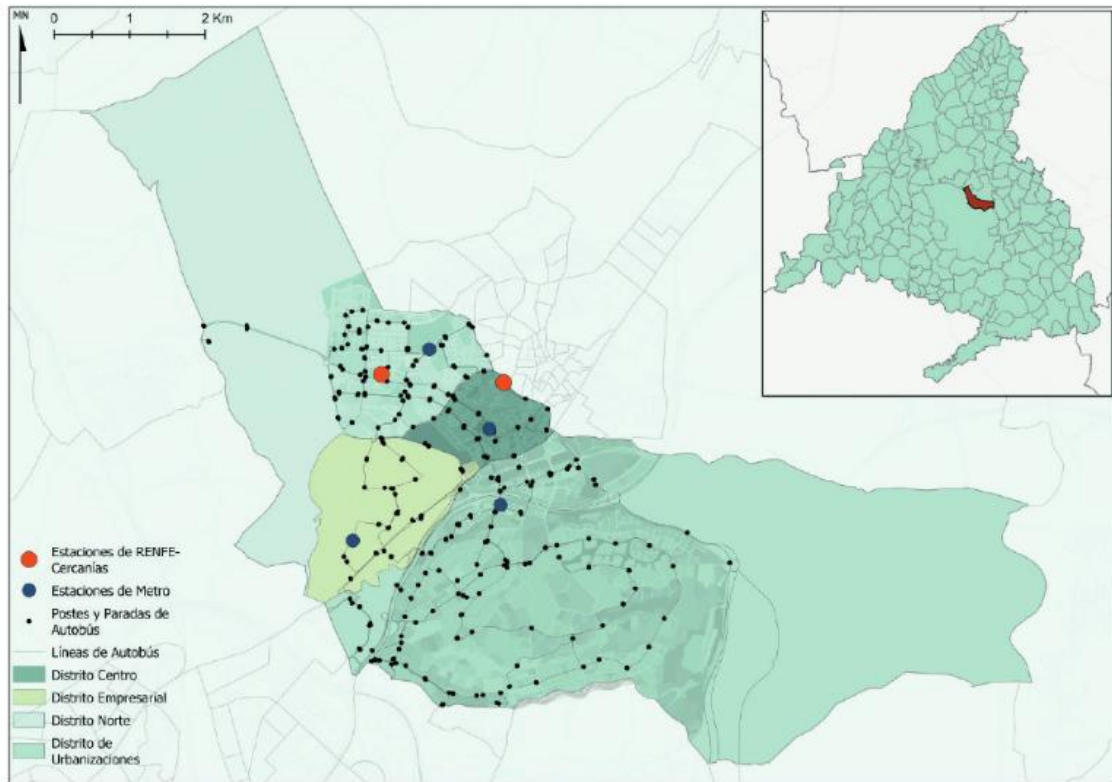


Ilustración 2 Ilustración del artículo de Roca y Moreno-Jimenez

2.2. Noticias sobre disponibilidad de servicios públicos en Madrid

- Falta de disponibilidad de centro de salud para consultas de pediatría en el municipio de San Martín de Valdeiglesias, obligando a sus vecinos a desplazarse 60km al centro de salud más cercano. [5]
- Una biblioteca por cada 101.000 madrileños [7]. Artículo que refleja la falta de disponibilidad de un servicio público del cual no se han construido instalaciones para satisfacer las necesidades de los madrileños.

3. MÉTODOS

La metodología utilizada en este trabajo se divide en distintas fases, empezando por la recopilación de datos hasta la obtención de las distancias finales y el análisis de resultados.

A grandes rasgos, los pasos realizados son los siguientes:

- Recopilación de los datasets georreferenciados con información sobre núcleos urbanos, delimitación de municipios, zonas de Dirección Asistencial de Salud o la posición de los servicios públicos sobre los que trata el trabajo.
- Procesado de los datos, añadiendo información adicional, estableciendo relaciones entre los distintos datasets, transformándolos o creando nuevos datos para poder ser utilizados a posteriori. Realizado mediante QGIS y Python
- Análisis de los datos, seleccionando para cada núcleo urbano, los servicios públicos más cercanos. Realizado a través de scripts de Python
- Cálculo de distancias de viaje, utilizando scripts de Python que realizan consultas a la API de Google Maps.
- Unión de los resultados obtenidos al dataset original: Los nuevos datos obtenidos son procesados y añadidos como atributos a los datasets, aportando información nueva a los datasets de centros urbanos y de municipios.

3.1. Fuente de los datos

La fuente principal de los datasets es la página web nomecalles del Instituto de

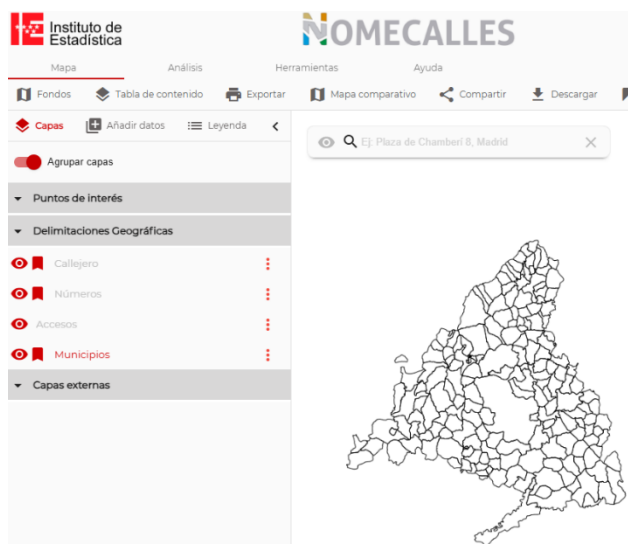


Ilustración 3 Captura de pantalla de la página web NOME CALLES

Estadística de la Comunidad de Madrid (IECM). La página contiene el nomenclátor oficial y distintas capas de información territorial con puntos de interés de titularidad pública.

A través de dicha página se obtuvieron los datasets de delimitaciones de los núcleos urbanos, las ubicaciones de los cuatro servicios públicos a analizar, junto con otros datasets que puedan ser útiles durante alguna otra fase del trabajo.

Otros datos adicionales, pero también imprescindibles en este trabajo se obtienen de otros sitios web como:

- Base de datos del Instituto Nacional de Estadística [5]
- Ministerio de Justicia de España [6]
- Consorcio Regional de Transportes de Madrid [7]

3.2. Procesamiento inicial de los datos mediante QGIS

Los datos obtenidos están en un formato legible por la herramienta de software QGIS [8]. Con esta herramienta de software podemos realizar distintas operaciones sobre los datos, añadir atributos con información extra o crear nuevas capas. Un ejemplo de esto es la adición de un atributo que indica, para cada núcleo urbano, la Dirección Asistencial de Salud en la que se encuentra, lo cual será necesario para determinar a qué hospital será referenciado.

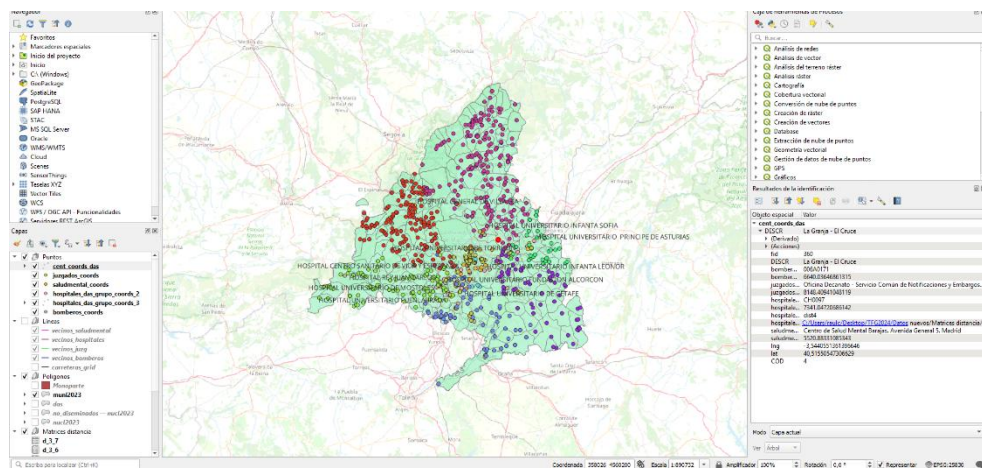


Ilustración 4 Vista previa de la herramienta QGIS

3.3. Procesamiento adicional mediante Python

El siguiente paso es determinar, para cada núcleo urbano, el hospital, parque de bomberos, juzgado y centro de salud mental más cercano. En un principio parecía posible hacerlo desde QGIS, pero este proceso en particular es muy tedioso y requiere una enorme cantidad de backtracking en caso de error. Por lo tanto, se decide utilizar la librería de QGIS para Python, la cual nos permite leer y operar sobre capas de

```
def get_nearest(project, origin, destinations):
    # Origen -> destinations dictionary
    relations = {}
    o_features = o_layer.getFeatures()
    for o_feat in o_features:
        relations[o_feat["DESCR"]] = dict.fromkeys(destinations)
    pairs = {}
    missing = {}
    cords_name = {}
    for i in range(len(destinations)):
        d_layer = project.mapLayersByName(destinations[i])
        d_layer = d_layer[0]
        o_features = o_layer.getFeatures()
        for o_feat in o_features:
            nearest_dist = 2000000
            nearest = None
            d_features = d_layer.getFeatures()
            for d_feat in d_features:
                if d_feat["COD"] != o_feat["COD"]:
                    continue
                ogeom = o_feat.geometry()
                dgeom = d_feat.geometry()
                dist = dgeom.distance(ogeom)
                if dist < nearest_dist:
                    nearest = d_feat
                    nearest_dist = dist
            if nearest is not None:
                if (o_feat["lat"], o_feat["lng"]) in pairs.keys():
                    pairs[o_feat["lat"], o_feat["lng"]].append((nearest["lat"], nearest["lng"]))
                else:
                    pairs[o_feat["lat"], o_feat["lng"]] = [(nearest["lat"], nearest["lng"])]
                relations[o_feat["DESCR"]][destinations[i]] = nearest["DESCR"]
            else:
                missing[o_feat["lat"], o_feat["lng"]] = destinations[i]
                relations[o_feat["DESCR"]][destinations[i]] = None
    out = PAIRS_PATH
    print(pairs)
    with open(out, 'w+') as outfile:
        pickle.dump(pairs, outfile)
    out = "C:/Users/raul/Desktop/2025/TF6/pickles/missing.pkl"
    with open(out, 'w+') as outfile:
        pickle.dump(missing, outfile)
    out = "C:/Users/raul/Desktop/2025/TF6/pickles/relations.pkl"
    with open(out, 'w+') as outfile:
        pickle.dump(relations, outfile)
    return
```

Ilustración 5 Vista previa del código usando la API de QGIS

De esta forma tenemos un método más intuitivo, automatizado y fácil de editar en caso de necesitar un cambio a posteriori, además de permitirnos generar nuevos archivos o realizar consultar en pasos posteriores.

3.4. Cálculo de distancias utilizando la API de Google Maps

La Google Maps Platform ofrece acceso a la Routes API [9] a usuarios registrados.

Haciendo uso de esta API se realizan consultas con un formato específico y obtenemos como resultado la distancia y tiempo exactos de un trayecto en coche a los conjuntos [origen, destino] que nosotros especifiquemos.

4. METODOLOGÍA PARA LA CONSTRUCCIÓN DEL DATASET

En este punto se explicará en más detalle los pasos realizados, así como los resultados obtenidos durante el proceso.

Cabe aclarar que aunque estas decisiones se presentan de forma lineal, algunas se toman en partes más tardías del desarrollo por motivo de backtracking. En estos casos, se aclarará el motivo de las decisiones o se dejará claro que se deben a etapas posteriores del desarrollo

4.1. Obtención de los datos iniciales

Nuestro punto de inicio es el sitio web NOMECALES. Como se describe en el punto 2.1., esta página nos permite visualizar y descargar distintos datasets georreferenciados de la Comunidad de Madrid, estos se dividen en Puntos y Delimitaciones, el primero ofrece datasets de puntos exactos y el segundo ofrece datasets de distintas divisiones territoriales.

De todos los conjuntos de datos que ofrece la página extraemos los siguientes:

- Delimitación de municipios: Delimitaciones → Municipios
- Núcleos urbanos: Delimitaciones → Nomenclátor → Núcleos
- Dirección Asistencial de Salud (DAS): Delimitaciones → Zonificación de Salud de Área Única → Dirección Asistencial de Salud
- Juzgados: Puntos → Admón. Pública y Seguridad Ciudadana → Instituciones Públicas → Admón. Justicia
- Centros de Salud Mental: Puntos → Salud y Servicios Sociales → Centros Sanitarios → Centros de Salud Mental
- Hospitales: Puntos → Salud y Servicios Sociales → Centros Sanitarios → Hospital
- Parques de bomberos: Puntos → Admón. Pública y Seguridad Ciudadana → Protección y Seguridad Ciudadana → Bomberos

Algunos datasets georreferenciados fueron obtenidos de otras fuentes al no aparecer en el nomenclátor de NOMECALES:

- Paradas de autobús EMT/Urbano/Interurbano: Obtenidas de la página del Consorcio de Transportes de Madrid [7]. Este dataset es necesario para obtener distancias utilizando transporte público
- Red de carreteras de Madrid: Versión modificada de dataset obtenido en el Centro de Descargas del Instituto Geográfico Nacional [10]. Este dataset es necesario para poder aproximar centroides a la carretera más cercana

Datos adicionales en formato tabla/CSV/XLSX

- Población por unidad poblacional: Obtenidas en el buscador del Insituto Nacional de Estadística [5]

4.2. Procesado de datos en QGIS

Una vez tenemos todos los datasets necesarios cargados en QGIS, podemos comenzar el proceso de transformación de los datos obtenidos.

Filtrado de diseminados y núcleos con menos de 10 habitantes

El primer paso a realizar es el filtrado del dataset de núcleos urbanos. En dicho conjunto de datos aparecen tanto núcleos de población como diseminados. El propio dataset contiene información sobre qué delimitación entra en cada una de estas dos categorías, por lo tanto, creamos una nueva capa que excluya todos los diseminados del conjunto.

A continuación, utilizando datos del INE sobre población por unidad poblacional [5], pasamos el dataset por otro filtrado, dejando fuera todos los núcleos poblacionales con menos de 10 habitantes.

Esta operación hace que pasemos de 1208 a 468 núcleos poblacionales sobre los que llevaremos a cabo nuestras operaciones

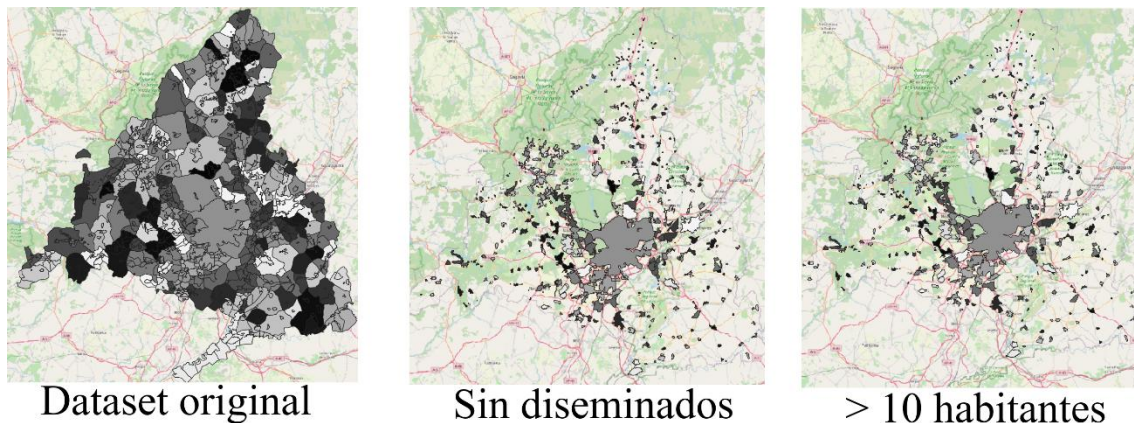


Ilustración 6 Proceso de filtrado de núcleos poblacionales

Obtención de los centroides

Como se puede observar, este dataset contiene datos georreferenciados de cada núcleo poblacional en forma de delimitaciones geográficas. Esta representación es importante pero no nos sirve para las operaciones que llevar a cabo. Lo que sí podemos utilizar es el centroide de cada uno de estos núcleos poblacionales, los cuales conservan además toda su información original: Nombre del núcleo, municipio al que pertenecen, población, etc.

Originalmente se planteó el uso de la herramienta Centroides, pero rápidamente se puede llegar a la conclusión de que muchos núcleos poblacionales tienen formas no convexas, haciendo que el centroide caiga fuera del núcleo.

Para evitar futuras complicaciones elegimos utilizar la herramienta Punto en superficie, la cual devuelve un dataset de puntos que, aunque no sean centroides de los núcleos poblacionales, está garantizado que se encuentra dentro de la delimitación de este.

En pasos posteriores se tuvo que hacer backtracking a este punto para añadir varios cambios al dataset que se necesitan para pasos posteriores:

- Mover cada centroide a la carretera más cercana, siempre y cuando esa carretera se encuentre dentro del mismo núcleo poblacional.
 - Para esta tarea se hace uso de un plugin de QGIS llamado ProcessX, el cual permite ajustar puntos a una línea añadiendo condiciones, en este caso, el de que pertenezca al mismo núcleo poblacional, evitando que se ajuste a carreteras de otro núcleo.

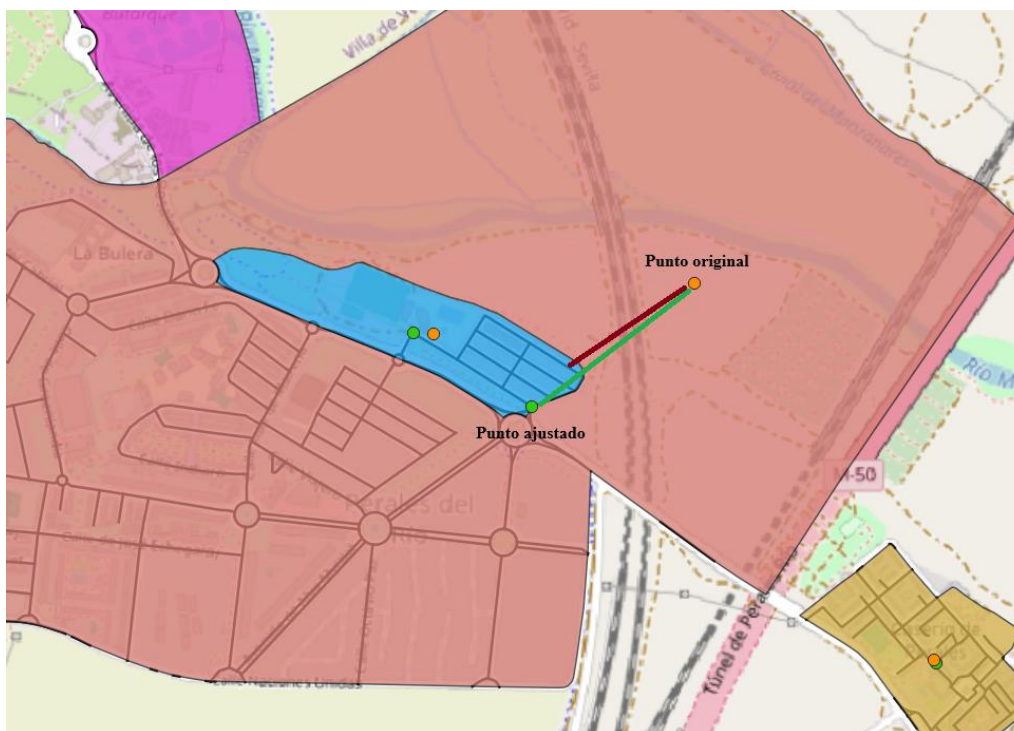


Ilustración 7 Ajuste de centroide a carretera

- Mover a mano ciertos núcleos poblacionales: Alrededor de 10 núcleos poblacionales se encuentran prácticamente incomunicados en lo que respecta a transporte público. En pasos posteriores, utilizaremos una API que devuelve distancias por transporte público, esta API tiene un grado de tolerancia limitado y puede calcular el tiempo a pie a la parada más cercana, pero si la distancia es excesiva deja de devolver resultados. Por lo tanto, movemos a mano estos centroides a la parada de bus más cercana, llevando un registro para tener en cuenta este ajuste en pasos posteriores.

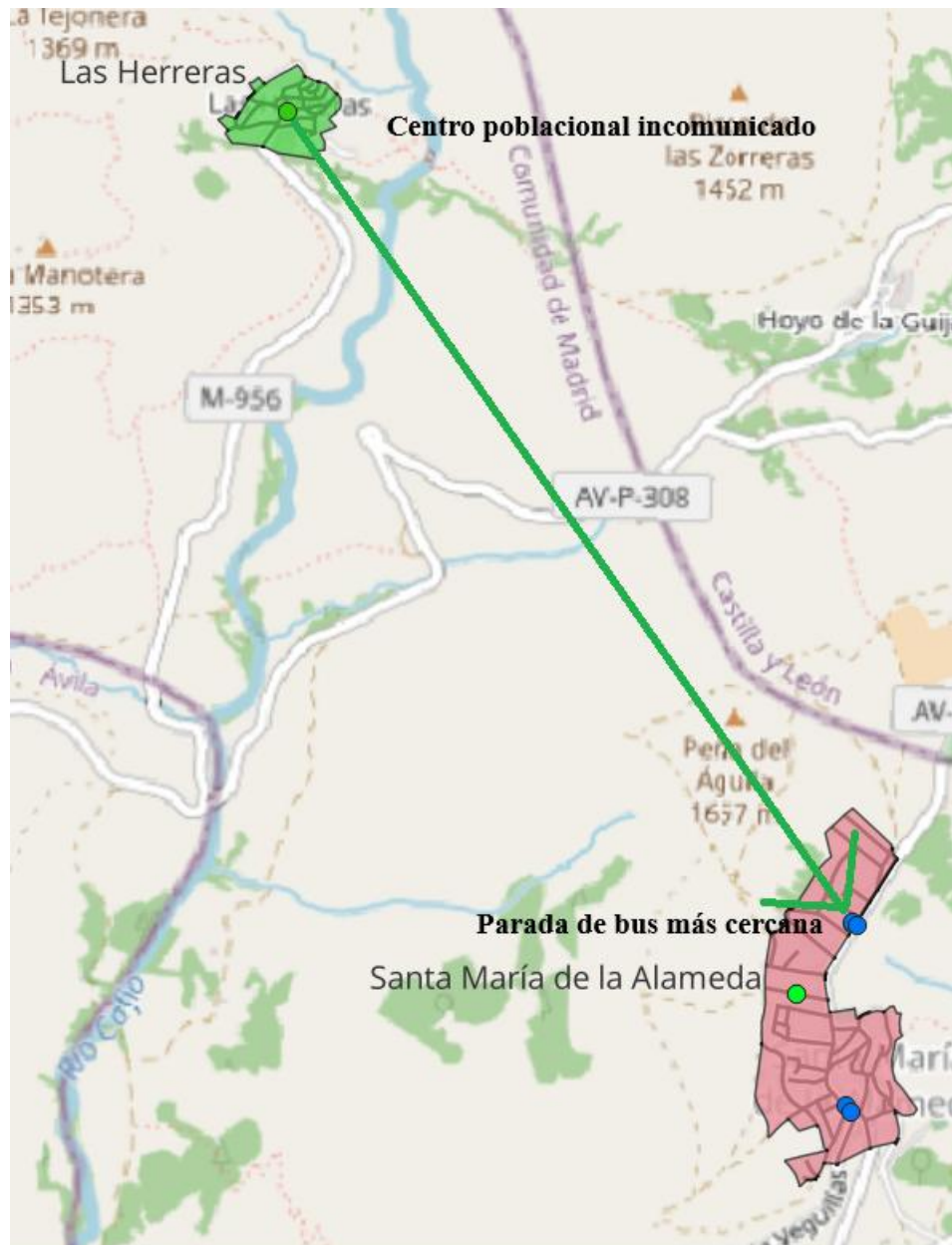


Ilustración 8 Ajuste a parada de bus más cercana

Filtrado de hospitales

El dataset de hospitales proporcionado por NOMEALLES contiene centros hospitalarios con enorme disparidad de tamaño y complejidad. Por lo tanto, no podemos considerar de la misma manera un núcleo de población cuyo hospital más cercano es un hospital de la complejidad más baja y un núcleo de población a la misma distancia del hospital Gregorio Marañón.

Por esto mismo, dividimos el dataset en otros dos nuevos conjuntos de datos, esta división la hacemos acorde a la complejidad de los hospitales y, como aparece reflejada en el dataset de NOMEALLES, se hace de forma manual guiándonos por la información sobre centros hospitalarios de la Red del Servicio Madrileño de Salud de la página web de la Comunidad de Madrid [11].

Los dos datasets resultantes son:

- Hospitales de grupo 3: Hospitales de gran complejidad.
- Hospitales de grupo 2: Hospitales de complejidad intermedia

En la siguiente tabla se puede ver la lista de hospitales que pasaron el filtro y que, por lo tanto, serán utilizados en los siguientes pasos:

Nombre	Grupo	Zona
Hospital Universitario 12 de Octubre	3	Centro
Hospital Universitario de la Princesa	3	Centro
Hospital Universitario Fundación Jiménez Díaz	3	Noroeste
Hospital Clínico San Carlos	3	Noroeste
Hospital Universitario Puerta de Hierro Majadahonda	3	Noroeste
Hospital Universitario La Paz	3	Norte
Hospital Universitario Ramón y Cajal	3	Norte
Hospital General Universitario Gregorio Marañón	3	Sureste
Hospital Centro Sanitario de Vida y Esperanza	2	Centro
Hospital Universitario Príncipe de Asturias	2	Este
Hospital Universitario de Torrejón	2	Este
Hospital General de Villalba	2	Noroeste
Hospital Universitario Infanta Sofía	2	Norte
Hospital Universitario de Móstoles	2	Oeste
Hospital Universitario Fundación Alcorcón	2	Oeste
Hospital Universitario Fuenlabrada	2	Oeste
Hospital Rey Juan Carlos	2	Oeste
Hospital Universitario Severo Ochoa	2	Sur
Hospital Universitario de Getafe	2	Sur
Hospital Universitario Infanta Leonor	2	Sureste

Ilustración 9 Tabla de hospitales de grupo 2 y 3

Adición de atributos

El dataset en este momento contiene todos los destinos (hospitales, juzgados, parques de bomberos y centros de salud mental), los municipios y los núcleos poblacionales con sus respectivos centroides ajustados. Pero aún se necesitan varios atributos que faciliten los pasos posteriores, además de brindar información adicional que puede ser relevante.

El primer atributo adicional es la **Dirección Asistencial de Salud (DAS)**. En Madrid existen 7 DAS y estas delimitaciones determinan cuál es el hospital de referencia de cada núcleo poblacional, esto evita que se calcule la distancia de un núcleo poblacional a un hospital de una DAS distinta.

Cada DAS tiene, por lo menos, un hospital de grupo 2, por lo que el hecho de la existencia de un hospital de grupo 3 dentro de la misma DAS se considera como un punto a favor al calcular la disponibilidad de un núcleo poblacional.

Cada DAS se representa por un atributo **COD**, con un número del 1 al 7. La herramienta Unir atributos por localización permite que un punto/delimitación de una capa herede el atributo de la delimitación de otra capa siempre que lo interseque o lo contenga. En la Ilustración 10 se puede ver la delimitación de cada DAS y cómo los núcleos y hospitales dentro de cada uno conserva el atributo COD de dicho DAS.

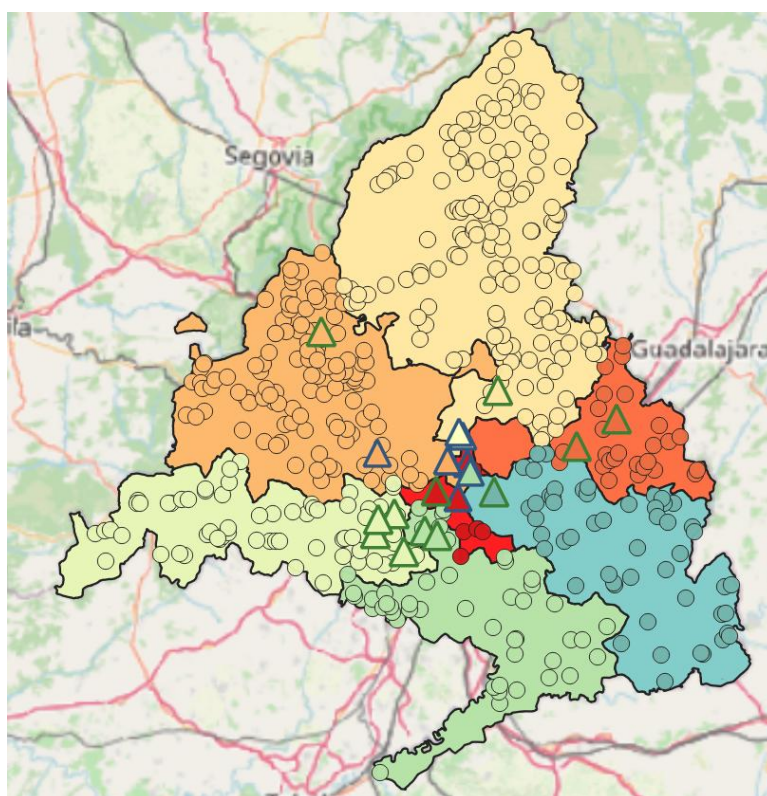


Ilustración 10 Núcleos y hospitales clasificados por su DAS

El segundo atributo adicional es el **partido judicial**. Según el Artículo 32 de la Ley Orgánica del Poder judicial [12], el partido judicial es una división de territorios uno o más municipios limítrofes, pertenecientes a una misma provincia. Esta división la

utilizaremos para que un núcleo urbano no sea dirigido a un juzgado de un partido judicial distinto.

Por lo tanto, se añade un atributo que determine a cuál de los 21 partidos judiciales pertenece cada núcleo de población y cada juzgado, acorde a los listados proporcionados por el Ministerio de Justicia [6] [13]. En la Ilustración 11 se representa una clasificación de núcleos poblacionales (círculos) y juzgados (cuadrados) por el partido judicial al que pertenecen.

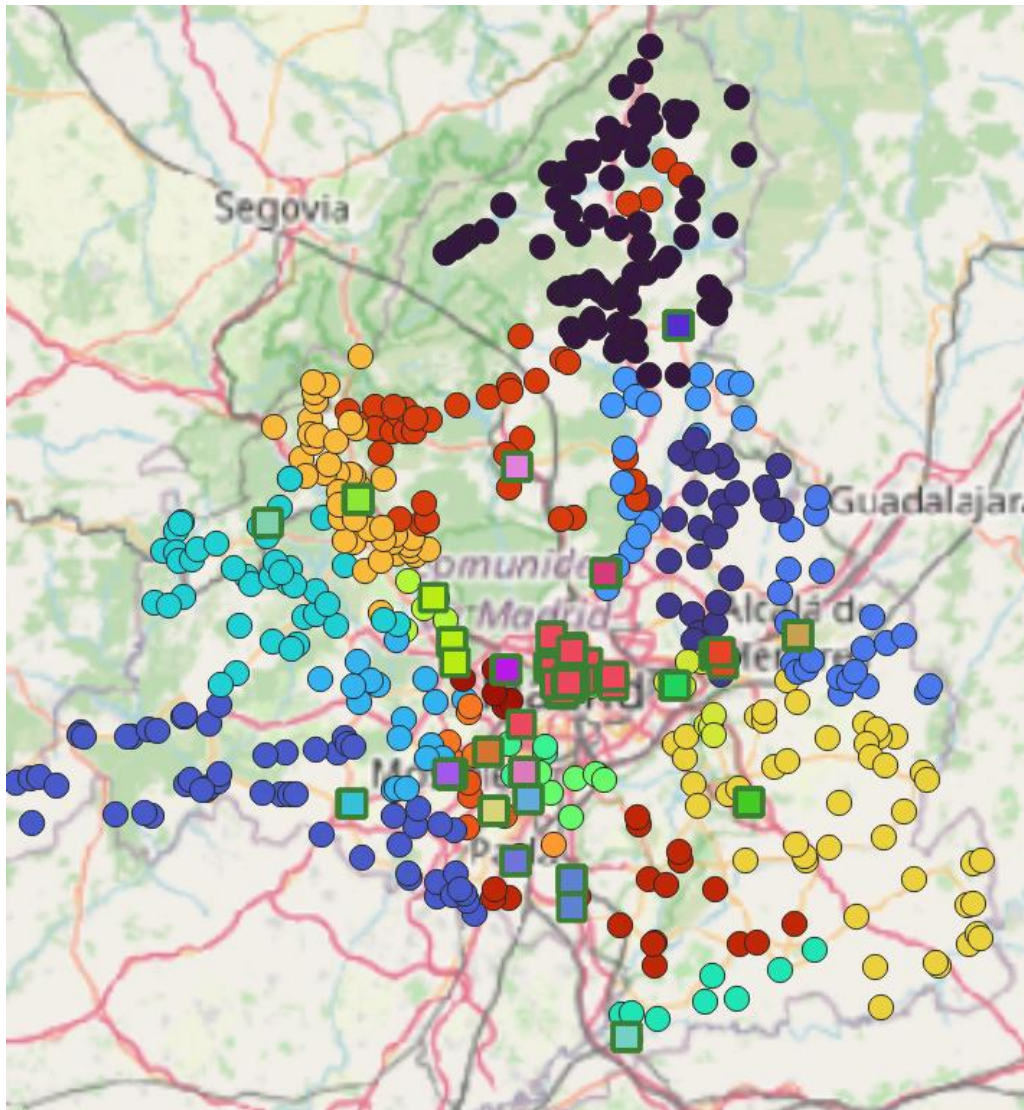


Ilustración 11 División por partido judicial

Atributos **lat** y **lng**. Contienen la latitud y longitud de cada punto. Estos atributos se pueden obtener en este punto del trabajo utilizando la herramienta Añadir campos X/Y a capa, pero se puede omitir completamente ya que utilizando la librería de QGIS para Python se puede obtener de forma rápida.

4.3. Selección de vecinos más cercanos a través de la librería de QGIS para Python

QGIS ofrece su propia librería de Python para realizar operaciones sobre capas de archivos .qgz, que contienen el proyecto QGIS completo.

El primer archivo Python utilizado se llama `file_creator.py` y contiene todos los métodos que permiten operar con archivos o leer el proyecto QGIS, permitiéndonos obtener los datos de todas las capas y operar sobre ellas.

Métodos `closest_destinations_cords()` y `closest_destinations_features()`

Esta función toma solo dos parámetros:

- **origin**: String con el nombre de la capa de origen, es decir, de los centroides de núcleos poblacionales.
- **destinations**: Lista con los nombres de las capas de hospitales, parques de bomberos, juzgados y centros de salud mental.

Antes de la ejecución de la función se definen varias variables.

`project` es, de forma resumida el proyecto QGIS donde se encuentran todas las capas.

El funcionamiento del algoritmo es, traducida a lenguaje natural:

1. Crea una lista con todos los orígenes y una lista para cada tipo de destino
2. Por cada núcleo itera por todos los destinos, y dependiendo del tipo de destino:
 - Hospitales: Guarda las coordenadas de todos los hospitales de grupo 2 y 3 por separado que pertenezcan al mismo DAS.
 - Juzgados: Filtra todos los juzgados que no estén en el mismo partido judicial y luego calcula la distancia euclídea a cada uno de ellos, guardando las coordenadas del juzgado más cercano.
 - Parques de bombero: No hace filtrado, por lo que calcula todas la distancia desde el núcleo a todos los parques de bomberos y guarda las coordenadas del más cercano.
 - Centros de salud mental: Filtra para cada núcleo los CSM del mismo DAS y calcula la distancia para quedarse con las coordenadas del más cercano
3. Devuelve un diccionario de Python que, para cada núcleo poblacional, contiene las coordenadas del propio núcleo y las coordenadas de los destinos.

```

def closest_destinations_cords(origin: str, destinations: list[str]) ->
dict[str,dict[str,list[str]]]:
    """
    Lee el proyecto QGIS y devuelve un diccionario
    Para cada origen guarda sus coordenadas y las coordenadas de los destinos más cercanos
    """
    closest = {}
    origin_layer = project.mapLayersByName(origin)[0]
    origin_features = list(origin_layer.getFeatures())
    destination_layers = [project.mapLayersByName(dest)[0] for dest in destinations]
    destination_features = [list(layer.getFeatures()) for layer in destination_layers]
    crs_src = origin_layer.crs()
    crs_dest = qgis.QgsCoordinateReferenceSystem("EPSG:4326")
    transform = qgis.QgsCoordinateTransform(crs_src, crs_dest, qgis.QgsProject.instance())
    for origin_feat in origin_features:
        origin_id = origin_feat[identifiers[origin_layer.name()]]
        origin_point = origin_feat.geometry().asPoint()
        origin_ll = transform.transform(origin_point)
        origin_cords = [origin_ll.y(),origin_ll.x()]
        closest[origin_id]={"cords":origin_cords, "destinations":[]}
        for i, dest_layer_feats in enumerate(destination_features):
            dest_name = destinations[i]
            filtered_dest_feats = []
            # Filtro
            if dest_name in ["Hospitales grupo 2", "Hospitales grupo 3", "Salud mental"]:
                if 'COD' in origin_feat.fields().names():
                    origin_cod = origin_feat['COD']
                    filtered_dest_feats = [f for f in dest_layer_feats if f['COD'] ==
origin_cod]
            elif dest_name == "Juzgados":
                if 'PART_JUD' in origin_feat.fields().names():
                    origin_part_jud = origin_feat['PART_JUD']
                    filtered_dest_feats = [f for f in dest_layer_feats if f['PART_JUD'] ==
origin_part_jud]
            else:
                filtered_dest_feats = dest_layer_feats

            if dest_name in ["Hospitales grupo 2", "Hospitales grupo 3"]:
                added = [[transform.transform(f.geometry().asPoint()).y(),
transform.transform(f.geometry().asPoint()).x()]
for f in filtered_dest_feats]
                closest[origin_id]["destinations"].append(added)
                continue
            # Busca el destino más cercano
            min_dist = float('inf')
            min_feat = None
            for dest_feat in filtered_dest_feats:
                dist = qgis.QgsDistanceArea().measureLine(
                    origin_feat.geometry().centroid().asPoint(),
                    dest_feat.geometry().centroid().asPoint()
                )
                if dist < min_dist:
                    min_dist = dist
                    min_feat = dest_feat
            # Guardamos sus identificadores
            pt = transform.transform(min_feat.geometry().asPoint())
            closest[origin_id]["destinations"].append([pt.y(), pt.x()])
    return closest

```

Código 1 Método closest_destinations_features()

El método closest_destinations_features() hace exactamente lo mismo pero el diccionario guarda los nombres de los destinos y orígenes. Es útil principalmente a la hora de comprobar que funciona correctamente

```

        # Guardamos sus identificadores
        closest_feats.append(min_feat[identifiers[dest_name]] if min_feat else
None)
        closest[origin_id] = closest_feats
    return closest

```

Código 2 Método closest_destinations_cords()


```

"0790101": {
  "cords": [
    40.42323536185605,
    -3.6629893725735445
  ],
  "destinations": [
    [
      40.43448605632367,
      -3.6758233245532175
    ],
    [
      40.37645105789588,
      -3.6968357352994903
    ],
    [
      40.38818516459446,
      -3.745274506608776
    ],
    [
      40.42855384810598,
      -3.667252259059084
    ],
    [
      40.43006735942887,
      -3.675012784351704
    ],
    [
      40.42153900790907,
      -3.6713676135003244
    ]
  ]
}

```

Ilustración 12 Fragmento del output de `closest_dest_cords()`

```

"0790101": [
  "HOSPITAL UNIVERSITARIO DE LA PRINCESA",
  "HOSPITAL UNIVERSITARIO 12 DE OCTUBRE",
  "HOSPITAL CENTRO SANITARIO DE VIDA Y ESPERANZA",
  "Parque de Bomberos de Madrid N\u00ba 2. Salamanca. Centro Coordinador",
  "Fiscal\u00eda ante el Tribunal Constitucional Calle Ortega y Gasset, 57. Madrid",
  "CSM Salamanca"
]

```

Ilustración 13 Fragmento del output de `closest_dest_features()`

4.4. Consulta a la Routes API de Google

Una vez tenemos la lista completa con cada uno de los núcleos poblacionales relevantes para el estudio, planteamos un plan de consultas a la Routes API de Google Maps Platform [9], esta API devuelve, entre otras cosas, la distancia real entre dos puntos.

Para obtener un resultado más preciso, para cada par origen-destino, pedimos que devuelva la distancia y tiempo de recorrido con estos parámetros:

- En hora punta (8:30) en coche
- En hora punta en transporte público
- En hora con menos tráfico (11:00) en coche

- En hora con menos tráfico en transporte público

Con estos cuatro resultados nos aseguramos de que se tienen en cuenta distintas situaciones que también representan la disponibilidad de un servicio público.

Métodos para la realización de consultas

En el archivo `google_request.py` se encuentra cualquier método relacionado con consultas a la API. Después de varias iteraciones de código llegamos a la conclusión de que ejecutar las consultas de forma asíncrona, dado que el número total de consultas a realizar se acerca a 12.000 y en algunos casos la API fallaba o había problemas de comunicación.

Los dos métodos principales del programa son `request_routes_v2_async()` y `fetch_route()`.

`request_routes_v2_async()` toma un solo parámetro booleano “preserve”, el cual, si es `True`, indica si sólo se realizan las consultas de las cuales no se tienen resultados todavía, permitiendo que un fallo de conexión puntual no nos obligue a realizar todas las consultas de nuevo. El resto del método hace llamadas asíncronas a la función `fetch_route()` y almacena y ordena los resultados de dicha función en forma de diccionario para ser utilizado a posteriori.

`fetch_route()`, en cambio, es el método que lleva a cabo las consultas, prepara la cabecera y cuerpo de cada consulta, notifica en caso de recibir una respuesta correcta o un mensaje de error y devuelve dicho resultado, así como el ID del origen, el ID del destino, el modo (en coche o transporte público) y la hora de la consulta, todo necesario para ensamblar los resultados en la función `request_routes_v2_async()`.

```

async def request_routes_v2_async(preserve=False):

    path = Path("output/closest_destinations_cords.json")
    with open(path, 'r') as file:
        json_file = json.load(file)

    if preserve:
        with open("output/routes_API_results_dump.json", 'r') as file:
            full_result = json.load(file)
    else:
        full_result = {}

    tasks = []

    for origin in json_file:
        if not preserve:
            full_result[origin] = []

        origin_cords = json_file[origin]["cords"]
        destinations = []
        for item in json_file[origin]["destinations"]:
            if item:
                if isinstance(item[0], list):
                    destinations.extend(item)
                else:
                    destinations.append(item)

        for dest_index, destination in enumerate(destinations):
            for mode in modes:
                for hour_index, hour in enumerate(hours):
                    if preserve:
                        if not
full_result[origin][dest_index][f"{mode}_{hours_true[hour_index].strftime("%H:%M")}"]:
                            tasks.append(
                                fetch_route(origin_cords, destination, mode, hour, origin,
dest_index)
                            )
                    else:
                        tasks.append(
                                fetch_route(origin_cords, destination, mode, hour, origin,
dest_index)
                        )

        results = await asyncio.gather(*tasks)

        # Reconstruir el resultado en estructura original
        for origin_id, dest_index, mode, hour, data in results:
            hour_key = hours_true[hour_index].strftime('%H:%M')
            key = f"{mode}_{hour_key}"

            if len(full_result[origin_id]) <= dest_index:
                full_result[origin_id].extend([{}]) * (dest_index - len(full_result[origin_id]) +
1))

            full_result[origin_id][dest_index][key] = data

    return full_result

```

Código 3 Método request_routes_v2_async

```

async def fetch_route(origin_cords, destination, mode, hour, origin_id, dest_index):
    async with semaphore:
        body = {
            "origin": {"location": {"latLng": {"latitude": origin_cords[0], "longitude":
origin_cords[1]}},
            "destination": {"location": {"latLng": {"latitude": destination[0], "longitude":
destination[1]}},
            "travelMode": mode.upper(),
            "departureTime": hour.strftime("%Y-%m-%dT%H:%M:%SZ")
        }
        if mode == "drive":
            body["routingPreference"] = "TRAFFIC_AWARE"

        for attempt in range(3): # Reintenta hasta 3 veces
            try:
                async with httpx.AsyncClient(timeout=30) as client:
                    print(f"Enviando {origin_id}, Destino {dest_index} [{mode}
{hour.strftime('%H:%M')}]")
                    response = await client.post(URL, headers=HEADERS, json=body)
                    response.raise_for_status()
                    print(f"Recibido: {origin_id}, Destino {dest_index} [{mode}
{hour.strftime('%H:%M')}]")
                    return (origin_id, dest_index, mode, hour, response.json())
            except httpx.HTTPStatusError as e:
                print(f"HTTP {e.response.status_code} en {origin_id}-{dest_index}-{mode},
intento {attempt+1}")
                await asyncio.sleep(1)
            except Exception as e:
                print(f"Error en {origin_id}-{dest_index}-{mode}: {e}")
                await asyncio.sleep(2)
        return (origin_id, dest_index, mode, hour, {"error": "intentos agotados"})

```

Código 4 Método fetch_route()

El resultado obtenido es un diccionario que fácilmente se puede convertir en archivo JSON con la siguiente estructura:

```

{
    "0790101": [
        {
            "drive_08:30": {
                "routes": [
                    {
                        "distanceMeters": 1929,
                        "duration": "464s"
                    }
                ]
            }, ← Aquí termina el resultado en coche a las 8:30
            ...
        }, ← Aquí termina el primer destino
        ...
    ]
}

```

4.5 Depurado de los resultados

Una vez se tienen los resultados en formato .json hace falta aplicarle varias capas de modificaciones para obtener lo que consideraríamos el dataset completo.

En el punto 4.2 mencionamos que ciertos núcleos tienen que ser movidos a una parada de bus cercana para poder obtener resultados en las consultas a Routes API. Pues ahora es el momento en el que es necesario tener en cuenta qué núcleos fueron desplazados y cuánto desplazamiento se le aplicó.

El desplazamiento se aplicó en una copia de la capa original, así que, teniendo ambas capas podemos calcular la distancia entre el mismo punto en estas dos capas obteniendo una distancia euclídea. Esto lo obtenemos fácilmente con las funciones `get_penalization()` y `apply_penalization()`

El funcionamiento general de estas funciones es: `get_penalization()` guarda todos los núcleos desplazados y su desplazamiento en distancia euclídea, `apply_penalizaciones()` toma esas penalizaciones y las aplica a todos los núcleos afectados.

Esta penalización es una aproximación y simula el camino andando entre la parada de bus y el destino, dado que la distancia euclídea no es una distancia realista, aproximamos que el camino real es aproximadamente un 150% de la distancia euclídea, ya que no es un camino recto, por lo que en el caso de la distancia, añadimos la penalización*1.5, mientras que para la duración del trayecto, ya que la velocidad media de una persona andando es de 1.4 m/s y asumimos que el trayecto es 1.5 veces la distancia euclídea, añadimos la penalización*1.5/1.4, o sea penalización*~1.07.

```
def get_penalization(old, new):
    old_layer = project.mapLayersByName(old)[0]
    old_features = list(old_layer.getFeatures())
    new_layer = project.mapLayersByName(new)[0]
    new_features = list(new_layer.getFeatures())
    penalizations = {}
    for old_feature in old_features:
        for new_feature in new_features:
            if old_feature["CDTNUCLEO"] == new_feature["CDTNUCLEO"]:
                dist = QgsDistanceArea().measureLine(
                    old_feature.geometry().centroid().asPoint(),
                    new_feature.geometry().centroid().asPoint()
                )
                if dist > 0:
                    penalizations[old_feature["CDTNUCLEO"]] = dist
    return penalizations

def apply_penalization(objective, pen_json):
    penalizations = json_to_dict(pen_json)
    results = json_to_dict(objective)
    for centro_key, penalization in penalizations.items():
        for result_key, result in results[centro_key].items():
            if "transit" in result_key:
                if "dis" in result_key:
                    results[centro_key][result_key] = result + int(penalization*1.5)
                if "dur" in result_key:
                    results[centro_key][result_key] = result + int(penalization*1.07)
    return results
```

Código 5 Métodos `get_penalization()` y `apply_penalization()`

El resultado de estos métodos es un nuevo archivo JSON con un formato mucho más legible que nos permite operar sobre el dataset de forma más sencilla. Para cada núcleo poblacional, podemos ver en la Ilustración 14 que se almacenan todos los datos relacionados con dicho núcleo con la notación

“<dis/dur>_<destino>_<modo>_<hora>”. Los datos de distancia se representan en metros y los de duración en segundos. En caso de los hospitales, se lleva un conteo del número de hospitales de los que se tiene información, por lo que <destino> pasa a ser “hospt<grupo>_<número>”

```
1  "0790101": {
2    "dis_hospt3_1_drive_08:30": 1929,
3    "dur_hospt3_1_drive_08:30": 464,
4    "dis_hospt3_1_drive_11:00": 1929,
5    "dur_hospt3_1_drive_11:00": 476,
6    "dis_hospt3_1_transit_08:30": 2174,
7    "dur_hospt3_1_transit_08:30": 1063,
8    "dis_hospt3_1_transit_11:00": 2213,
9    "dur_hospt3_1_transit_11:00": 1317,
10   "dis_hospt3_2_drive_08:30": 8244,
11   "dur_hospt3_2_drive_08:30": 793,
12   "dis_hospt3_2_drive_11:00": 8244,
13   "dur_hospt3_2_drive_11:00": 835,
14   "dis_hospt3_2_transit_08:30": 7175,
15   "dur_hospt3_2_transit_08:30": 2281,
16   "dis_hospt3_2_transit_11:00": 7175,
17   "dur_hospt3_2_transit_11:00": 2341,
18   "dis_hospt2_1_drive_08:30": 14163,
19   "dur_hospt2_1_drive_08:30": 1467,
20   "dis_hospt2_1_drive_11:00": 14163,
21   "dur_hospt2_1_drive_11:00": 1489,
22   "dis_hospt2_1_transit_08:30": 12782,
23   "dur_hospt2_1_transit_08:30": 3163,
24   "dis_hospt2_1_transit_11:00": 12782,
25   "dur_hospt2_1_transit_11:00": 3223,
26   "dis_juzgados_drive_08:30": 1043,
27   "dur_juzgados_drive_08:30": 253,
28   "dis_juzgados_drive_11:00": 1043,
29   "dur_juzgados_drive_11:00": 281,
30   "dis_juzgados_transit_08:30": 1242,
31   "dur_juzgados_transit_08:30": 702,
32   "dis_juzgados_transit_11:00": 825,
33   "dur_juzgados_transit_11:00": 754,
34   "dis_bomberos_drive_08:30": 1633,
35   "dur_bomberos_drive_08:30": 403,
36   "dis_bomberos_drive_11:00": 1633,
37   "dur_bomberos_drive_11:00": 455,
38   "dis_bomberos_transit_08:30": 1827,
39   "dur_bomberos_transit_08:30": 1052,
40   "dis_bomberos_transit_11:00": 1652,
41   "dur_bomberos_transit_11:00": 1166,
42   "dis_csmental_drive_08:30": 1582,
43   "dur_csmental_drive_08:30": 402,
44   "dis_csmental_drive_11:00": 1582,
45   "dur_csmental_drive_11:00": 458,
46   "dis_csmental_transit_08:30": 1293,
```

Ilustración 14 Ejemplo de JSON despues de penalizaciones

4.6. Unión con el dataset en formato Geopackage

Una vez tenemos todos los datos en formato JSON está todo preparado para el siguiente paso: unir los resultados obtenidos con las capas de proyecto QGIS.

Este proceso es quizás el más largo dentro del procesamiento de datos y las funciones implementadas son demasiado largas como para ser presentadas, pero esta es una

descripción de los pasos a seguir, además de proveer un vistazo de los resultados obtenidos.

Capa de núcleos poblacionales

Primero fusionamos los resultados obtenidos con la capa de centroides de los núcleos poblacionales. Debido a que cada núcleo poblacional tiene un número distinto de hospitales cercanos:

- El máximo de hospitales de grupo 2 cercanos es 4, por lo que cada núcleo poblacional tendrá columnas para información de 4 hospitales de grupo 2, si un núcleo en específico tiene menos, las columnas toman valor nulo.
- El máximo de hospitales de grupo 3 cercanos es 3, se toma la misma medida que en el punto anterior.

Además de estos datos añadimos varios atributos extra:

- Media de distancia y duración a cada servicio público (10 columnas)
- Media de las medias de distancia y duración a cada servicio público (2 columnas)

Por último, para facilitar el paso posterior añadimos 12 nuevas columnas a partir de multiplicar las 12 columnas anteriores por la población de cada núcleo poblacional. Esto nos permitirá en el paso posterior calcular una media ponderada que tenga en cuenta la población de cada núcleo.

Antes de introducir las atributos añadidos, debemos explicar el significado de ciertas variables:

- `<tipo>` toma los valores:
 - “dis”: Si informa sobre la distancia (en metros)
 - “dur”: Si informa sobre la duración de viaje (en segundos)
- `<destino>` toma los valores:
 - “bomberos”:
 - “csmental”:
 - “juzgados”:
 - `hospt<3|2>_<n>`: Como se explica al principio de este punto. Puede haber datos de varios hospitales de cada tipo. En caso de ser el hospital de grupo 2 número 3 sería “hospt2_3”
- `<modo>` toma los valores:
 - “drive”: Para trayectos en carretera
 - “transit”: Para trayectos en transporte público
- `<hora>` toma los valores:
 - “08:30” para trayectos en hora punta
 - “11:00” para trayectos con tráfico relajado

Una vez explicado esto, aquí podemos ver un desglose de los atributos añadidos al dataset de núcleos poblacionales a partir de los resultados obtenidos de Routes API:

Atributo	Descripción
<tipo>_<destino>_<modo>_<hora>	Esta descripción abarca un total de 80 columnas con los datos originales obtenidos de Routes API, algunos con modificaciones debido a las penalizaciones aplicadas en el punto 4.5 Depuración de datos
mean_<tipo>_<destino>_<modo>	20 columnas. La media de distancia o duración para cada tipo de servicio público en coche o transporte público
mean_<tipo>_<destino>_all	10 columnas. La media de distancia o duración por cada tipo de servicio público
mean_<tipo>_<modo>	4 columnas: • mean_dis_drive: Media de todas las distancias en coche • mean_dis_transit: Media de todas las distancias en transporte público • mean_dur_drive: media de todas las duraciones en coche. • Mean_dur_tranist: Media de todas las duraciones en transporte público
mean_<tipo>_all	2 columnas: • mean_dis_all: Media de todas las distancias saliendo del núcleo • mean_dur_all: Media de todas las duraciones de trayecto saliendo del núcleo
weigh_mean....	36 columnas, cada una de las columnas que comienzan por “mean” multiplicadas por la población del núcleo. Útil para el cálculo de la media ponderada en la capa de municipios

Esto significa que se añaden 152 nuevos atributos al dataset de núcleos poblacionales que, junto a los 12 anteriores, nos deja con un dataset con información sobre 468 núcleos poblacionales y 164 atributos.

Capa de municipios

A la hora de crear la capa de núcleos poblacionales nos aseguramos de crear atributos con datos útiles para calcular la media ponderada en esta capa. Estos valores los llamaremos valores ponderados y sumaremos todos los valores ponderados de cada núcleo dentro de cada municipio y lo dividiremos por la población total de cada municipio.

$$valor_{pond} = valor * poblacion,$$

$$media_{pond} = \frac{\sum valor_{pond}}{\sum poblacion}$$

A partir de esta operación obtenemos 36 atributos con las medias ponderadas de atributos clave de los núcleos poblacionales de cada municipio, además de tener en cuenta la población de cada núcleo para obtener resultados más precisos.

Como resultado de esta operación y unir dichos atributos con la capa, damos terminado el procesamiento de datos

5. DESCRIPCIÓN DEL DATASET OBTENIDO

Metadatos

Indicadores Obtenidos

Formatos de publicación

6. ESPECIFICACIÓN DE REQUISITOS DE LA APLICACIÓN GIS PARA VISUALIZAR LOS DATOS

6.1 Presentación de los formalismos para registrar las Requisitos.

6.2 Especificación de Requisitos

6.3 Plan de Pruebas

7. DISEÑO

7.1 Diseño de la IU (Aplicación de Prueba)

7.1.1. Paper prototype o mock up inicial de la aplicación

7.1.2. Versión final de la UI

7.1.3 Diagramas de clases (si aplica)

7.2 Diseño de la arquitectura de servidor (Back end).

7.2.1 Alternativas (definir varias posibles alternativas y seleccionar la que más te convengan).

7.2.2. Diagrama de paquetes o componentes y clases de los elementos de la arquitectura de servidor.

8. RESULTADOS

Especificación de preguntas de análisis de la accesibilidad de los servicios públicos en la Comunidad de Madrid y su discusión con los datos estadísticos proporcionados por el dataset generado.

9. CONCLUSIONES Y FUTUROS TRABAJOS

Referencias

- [1 Instituto Nacional de Estadística, «Núcleo de población,» [En línea]. Available:
] <https://www.ine.es/DEFIne/es/concepto.htm?c=4930>. [Último acceso: 2025].
- [2 Instituto Nacional de Estadística, «Diseminado,» [En línea]. Available:
] <https://www.ine.es/DEFIne/concepto.htm?c=5188>.
- [3 Instituto de Estadística de la Comunidad de Madrid, «Nomecalles,» [En línea].
] Available: https://gestiona.comunidad.madrid/nomecalles_web/#/inicio.
- [4 «Datos Abiertos del Consorcio Regional de Transportes de Madrid,» 2025. [En
] línea]. Available: <https://data-crtm.opendata.arcgis.com/search>.
- [5 D. Gutiérrez, «Recorrer 60 Kilómetros Para Llevar a Tu Hijo al Pediatra: Así Se
] Vive En Algunos Pueblos de Madrid.,» *El País*, pp. elpais.com/espana/madrid/2024-12-07/recorrer-60-kilometros-para-llevar-a-tu-hijo-al-pediatra-asi-se-vive-en-algunos-pueblos-de-madrid.html, 7 12 2024.
- [6 Instituto Nacional de Estadística, «Nomenclátor. Población por unidad poblacional,»
] [En línea]. Available: <https://www.ine.es/nomen2/>.
- [7 Ministerio de justicia de España, «Nomenclátor de Partidos Judiciales de España,»
] [En línea]. Available:
https://www.mjusticia.gob.es/es/JusticiaEspana/OrganizacionJusticia/Documents/011223_Nomenc1%C3%A1tor%20de%20Partidos%20Judiciales.pdf.
- [8 QGIS.org, «Spatial without Compromise · QGIS Website,» QGIS Association,
] 2025. [En línea]. Available: <http://www.qgis.org>.
- [9 Google Maps Platform Documentation | Routes API | Google for Developers, [En
] línea]. Available: <https://developers.google.com/maps/documentation/routes>.
[Último acceso: 2025].
- [1 «Centro de Descargas del CNIG (IGN),» [En línea]. Available:
0] <https://centrodedescargas.cnig.es/CentroDescargas/redes-transporte>. [Último acceso: 2025].
- [1 Comunidad de Madrid, «Hospitales de la red del Servicio Madrileño de Salud,» [En
1] línea]. Available: <https://www.comunidad.madrid/servicios/salud/hospitales-red-servicio-madrileno-salud>.
- [1 Boletín Oficial del Estado, «Artículo 32 Ley Orgánica del Poder judicial,» [En
2] línea]. Available: <https://www.iberley.es/legislacion/articulo-32-ley-organica-poder-judicial>.

[1 Procuradora Asociados, «Buscador de PARTIDOS JUDICIALES EN Madrid,» [En 3] línea]. Available: <https://procuradoresasociadosmadrid.es/partidos-judiciales-madrid/>.